

SPEC-1-Auditoría Médica Web (GitHub Pages)

Background

La organización requiere una aplicación web (publicada en GitHub Pages) para soportar la **auditoría médica mensual** del registro clínico, alineada al procedimiento institucional MTR-GM-PR-007 y sus anexos. El flujo de alto nivel incluye: selección de la base mensual (ARC_AUDITORIA_MEDICA), cálculo y **estratificación de la muestra por especialidad** según la metodología del Anexo 2, **asignación equitativa** de historias clínicas (HC) a auditores, ejecución de auditorías en Reliv/SIMEC utilizando el **número de admisión**, calificación **binaria (1/0)** de criterios (Anexo 5), generación de **reporte y recomendaciones**, y construcción de una **base de retroalimentación**.

Desde el punto de vista de producto, se requiere un **dashboard general** con métricas operativas y clínicas, cortes por ciudad/especialidad/médico y una ficha por médico con su **score** e historias auditadas. Debe existir un **módulo de auditoría ultra-rápido** (calificación a un clic por criterio), **gestión de usuarios** con dos perfiles (Admin y Médico Auditor), **asignación manual de historias** por el Admin, **bitácora** (log) y un **módulo de carga de Base Pedidos semanal** para disparar **alertas** (reconsulta en 7 días por especialidad, umbral de medicamentos >5, laboratorio ≥ 5 , imagen ≥ 2).

Restricciones clave: hospedaje estático (GitHub Pages), foco en **funcionalidad/UI** inicial (estilo mínimo viable), y buenas prácticas de seguridad y privacidad (datos clínicos).

Requirements

Supuestos base

- **Hosting:** GitHub Pages (aplicación 100% frontend).
- **Backend-as-a-Service:** Firebase (Auth, Firestore, Storage, Functions/Extensions solo si se requieren notificaciones/automatizaciones).
- **Integración clínica:** por ahora **sin APIs** con Reliv/SIMEC (uso manual en pestaña separada).
- **Datos de entrada:** carga **manual** de archivos institucionales:
 - Mensual: `ARC_AUDITORIA_MEDICA` (CSV).
 - Semanal: `BASE_PEDIDOS` (CSV) para alertas.
 - Parámetros/criterios: "Modelo Auditoría Nuevo.xlsx" / Anexo 5 (XLSX) como catálogo de criterios.

MoSCoW

Must Have (imprescindible)

1. **Autenticación y perfiles:** Login con Firebase Auth; perfiles **Admin** y **Médico Auditor**; asignación automática de perfil según rol en Firestore.
2. **Carga de datos:** UI para cargar `ARC_AUDITORIA_MEDICA` (mensual) y `BASE_PEDIDOS` (semanal); validación de esquema; previsualización; control de versiones y sello de tiempo.
3. **Muestreo mensual:** Filtro por **mes** en `ARC_AUDITORIA_MEDICA`; **generación de muestra total** y **estratificación por especialidad** conforme a metodología del **Anexo 2**; registro de la "cohorte de auditoría" generada.
4. **Asignación equitativa:** Distribución **igualitaria** de HC estratificadas entre auditores con balanceo por carga (round-robin), con posibilidad de **asignación manual** por Admin.
5. **Módulo de auditoría 1-clic:** Búsqueda por **# de admisión**

(desde la muestra asignada); formulario ultrarrápido con **criterios binarios (1/0)** del **Anexo 5**; autosave; comentarios/recomendaciones por caso; marcar **finalizada**. 6. **Dashboard general** (filtrable por **ciudad, especialidad, médico, mes**): - **Indicadores** solicitados y metas/rangos: - Cumplimiento de calidad en el registro clínico (meta $\geq 85\%$). - Índice de eficiencia en toma de signos vitales (meta $\geq 90\%$). - Pertinencia de exámenes complementarios (meta $>90\%$). - Cumplimiento de guías de práctica clínica (meta $\geq 90\%$). - Cumplimiento de auditorías por médico auditor (meta $>95\%$). - Cumplimiento en retroalimentación y seguimiento (meta $\geq 90\%$). - **Cortes** por ciudad/especialidad/médico y **tarjeta de médico** (nombre, especialidad, centro médico, score global, lista de historias auditadas con # de admisión y calificación). - **Índices operativos mensuales**: reconsulta, # laboratorios, # imágenes. 7. **Alertas (Base Pedidos)**: - Reconsulta: paciente con >1 cita **misma especialidad** dentro de **7 días**. - Umbrales: **medicamentos >5 , laboratorio ≥ 5 , imagen ≥ 2** . - Bandeja de alertas y marcación de atención/seguimiento. 8. **Retroalimentación**: repositorio de **hallazgos** por historia/criterio, estado de retroalimentación y trazabilidad. 9. **Admin**: - **Gestión de usuarios** (crear/editar/eliminar, reset de contraseña, asignación de rol). - **Asignar historias** (manual y re-asignar). - **Catálogos**: ciudades, especialidades, centros, auditores. - **Bitácora** (log) de acciones sensibles (carga de datos, asignaciones, edición de auditorías). 10. **Seguridad/Privacidad**: reglas de seguridad de Firestore (least privilege), cifrado en tránsito, minimización de datos, control de acceso por rol y por **propietario de asignación**.

Should Have (debería tener) 1. Validadores de calidad de datos (tipos, columnas obligatorias, duplicados) y mapeo de **especialidades** a un catálogo estándar. 2. Balanceo de carga con **exclusiones** (conflicto de interés) y re-balanceo. 3. **Exportación** de reportes (CSV/PDF) y **scorecards** por médico/servicio. 4. UI **responsive**, accesible (WCAG AA), y soporte **offline básico** (caché de criterios y últimas asignaciones). 5. Notificaciones por correo/Slack para alertas y pendientes (vía Firebase Functions/Extensions).

Could Have (podría tener) 1. Configuración dinámica de **metas/umbrales** desde Admin. 2. Plantillas de recomendación y textos sugeridos por criterio. 3. Historial de versiones de una auditoría (diff). 4. Modo oscuro y temas.

Won't Have (por ahora) 1. Integración API con Reliv/SIMEC. 2. Diseño visual avanzado ("wow"). 3. Modelos predictivos/ML o benchmarking externo.

Method

1) Arquitectura

- **SPA en React + TypeScript** (Vite) hospedada en **GitHub Pages**.
- **Firebase**: Auth (Email/Password + SSO opcional), Firestore (datos), Storage (versionado de CSV/XLSX), Functions (tareas batch/cron de agregados y alertas), Hosting opcional para funciones.
- **Gestión de estado**: Zustand o Redux Toolkit (simple, serializable).
- **UI**: componentes accesibles (Radix UI/Headless UI), tablas (AG Grid o TanStack Table), gráficos (Chart.js o ECharts).
- **Ruteo**: React Router.
- **Librerías** auxiliares: Papaparse (CSV), xlsx (lectura XLSX), date-fns (fechas), zod (validación de esquemas).
- **CI**: GitHub Actions (lint, build, deploy a Pages).

Diagrama (componentes)

```

@startuml
package "GitHub Pages (Frontend)" {
    [React SPA]
    [UI Dashboard]
    [UI Auditoria 1-clic]
    [UI Admin]
}
package "Firebase" {
    [Auth]
    [Firestore]
    [Storage]
    [Functions]
}
[React SPA] --> [Auth]
[React SPA] --> [Firestore]
[React SPA] --> [Storage]
[Functions] --> [Firestore]
[Functions] --> [Storage]
@enduml

```

2) Modelo de Datos (Firestore)

Colecciones (prefijo `prod_` en producción; `dev_` en desarrollo):

- `cohortes` (una por mes):
 - `id` (yyyymm), `mes`, `estado` (borrador/final), `totales` {porEspecialidad, porCiudad}, `param_sampling` {seed, minPorEstrato, reglaRedondeo}.
- `historias` (instancias incluidas en la cohorte):
 - `id` (admisión), `cohorteId`, `pacienteId` (hash), `especialidad`, `ciudad`, `centro`, `fechaAdmision`, `asignadaA` (uid), `estado` (pendiente/en_proceso/finalizada), `auditoriaRef`.
- `criterios` (catálogo desde **Anexo 5**):
 - `id`, `seccion` (p.ej., Anamnesis, Examen Físico...), `nombre`, `descripcion`, `tipo` (binario), `peso` (default 1), `activo`.
- `auditorias`:
 - `id`, `historiaId`, `auditorId`, `cohorteId`, `fechaInicio`, `fechaCierre`, `respuestas` [{ `criterioId`, `valor` 0/1, `nota` }], `score` ($\sum \text{valor} * \text{peso} / \sum \text{peso}$), `recomendacion` (texto), `retroalimentacion` { `estado`: pendiente/enviada/cerrada, `fecha` }.
- `usuarios`:
 - `uid`, `rol` (admin/auditor), `nombre`, `especialidad`, `centro`, `ciudad`, `email`, `activo`.
- `pedidos_semana` (carga semanal `BASE_PEDIDOS`):
 - `id` (yyyymm), `rangoFechas`, `archivoRef`, `totales`, `procesado`.
- `alertas`:
 - `id`, `tipo` (reconsulta/meds/lab/imagen), `pacienteId` (hash), `especialidad`, `fecha`, `detalle`, `estado` (nueva/en_atencion/cerrada), `cohorteId?`.

- **agregados** :
- **id** (cohorteId o cohorteId-especialidad o cohorteId-medico), **dimension** (global/ciudad/especialidad/medico), **metricas** {ver fórmulas}, **umbral** {meta por indicador}.
- **logs** :
- **id**, **actorUid**, **accion**, **entidad**, **referenciaId**, **timestamp**, **payload** (parcial), **ip?**.

Índices sugeridos - **historias**: cohorteId+especialidad, asignadaA+estado, ciudad+especialidad. - **auditorias**: cohorteId+auditorId, historiaId, score (para ordenamientos). - **alertas**: tipo+estado, especialidad+fecha. - **agregados**: dimension+id.

Diagrama (modelo lógico)

```
@startuml
class Cohorte { id mes estado param_sampling totales }
class Historia { id cohorteId pacienteId especialidad ciudad centro
fechaAdmision asignadaA estado auditoriaRef }
class Criterio { id seccion nombre descripcion tipo peso activo }
class Auditoria { id historiaId auditorId cohorteId fechaInicio fechaCierre
score }
class Usuario { uid rol nombre especialidad centro ciudad email activo }
class PedidoSemana { id rangoFechas archivoRef procesado }
class Alerta { id tipo pacienteId especialidad fecha estado }
class Agregado { id dimension metricas umbral }
Cohorte "1" -- "*" Historia
Historia "1" -- "0..1" Auditoria
Auditoria "*" -- "*" Criterio : respuestas{valor}
Usuario "1" -- "*" Auditoria
PedidoSemana "1" -- "*" Alerta
Agregado .. Cohorte
@enduml
```

3) Ingesta de Datos y Esquemas

Contratos (estrictos, validación con Zod): - **ARC_AUDITORIA_MEDICA (1).csv** (mensual) — columnas mínimas: - **admission_id** (string), **patient_document** (string u ofuscado), **patient_id_hash** (sha256 del doc), **especialidad**, **ciudad**, **centro**, **fecha_admision** (YYYY-MM-DD), **medico_tratante** (nombre opcional), **diagnostico_principal** (CIE10 opcional). - **BASE_PEDIDOS.csv** (semanal): - **patient_document**, **patient_id_hash**, **fecha** (YYYY-MM-DD), **especialidad**, **meds_count** (int), **labs_count** (int), **images_count** (int), **ciudad**, **centro**. - **Modelo Auditoría Nuevo.xlsx** (Anexo 5): - Hoja **critérios**: **criterio_id**, **seccion**, **nombre**, **descripcion**, **peso** (default 1), **activo** (bool).

Proceso de carga (Admin) 1. Subir archivos → **Storage** (carpeta **/raw/yyyymm/** o **/raw/yyyyww/**). 2. UI valida esquema y previsualiza diferencias. 3. Escribir metadatos y registros en **Firestore** (**cohortes**,

historias, pedidos_semana). 4. Ejecutar **Function** para pre-procesar (agregados iniciales, alertas).

4) Muestreo y Estratificación (Anexo 2)

Filtro por mes: seleccionar registros de ARC_AUDITORIA_MEDICA con fecha_admision en el mes objetivo.

Muestra total: por defecto **todas** las HC del mes. Opcional: parámetro sample_ratio (0-1) y/o tamano_muestra si en el Anexo 2 se define un tamaño; si no se especifica, usar total.

Estratificación por especialidad: 1. Agrupar por especialidad → tamaños N_e . 2. Definir tamaño de muestra n (total) y calcular $n_e = \text{round}(N_e / \sum N_e * n)$ usando **método de restos mayores** para ajustar al total; garantizar minPorEstrato (≥ 1 si $N_e > 0$). 3. **Aleatorización reproducible** dentro de cada estrato: seed = yyyy-mm y hash(admission_id + seed) para ordenar y tomar los primeros n_e .

Asignación equitativa a auditores: - Input: lista de auditoresActivos, exclusiones opcionales (conflicto de interés por centro/especialidad), carga previa. - Algoritmo **round-robin balanceado** por especialidad: iterar por estratos y asignar en orden cíclico manteniendo conteos por auditor; si hay exclusión, saltar al siguiente.

Pseudocódigo

```
for cada especialidad in estratos:
    muestra_e = sample(orden_random, n_e)
    for hc in muestra_e:
        auditor = siguienteAuditorBalanceado(especialidad)
        asignar(hc, auditor)
```

5) Flujo del Auditor (UI 1-clic)

- El auditor ve su **bandeja** (pendientes/en proceso/finalizadas).
- Abre una HC → botón "Abrir en Reliv" (link externo) usando el admission_id.
- Sección **Anamnesis** primero; cada criterio (binario) tiene **toggle 1/0** a un clic; atajos de teclado; autosave cada cambio.
- Se calcula **score** en tiempo real ($\sum \text{valorpeso} / \sum \text{peso}$). *Campo de recomendación** libre.
- **Cerrar auditoría** (confirmación); movido a "finalizadas".

Diagrama (secuencia)

```
@startuml
actor Auditor
Auditor -> SPA: Login
SPA -> Auth: authenticate()
```

```

Auth --> SPA: token
Auditor -> SPA: Abrir bandeja
SPA -> Firestore: query(historias asignadas)
Auditor -> SPA: Abrir HC(admission_id)
SPA -> Browser: window.open(Reliv, admission_id)
Auditor -> SPA: Marcar criterios 1/0
SPA -> Firestore: upsert(auditoria.respuestas, autosave)
SPA -> Firestore: update(historias.estado="en_proceso")
Auditor -> SPA: Cerrar auditoría
SPA -> Firestore: update(auditoria.score, fechaCierre)
SPA -> Firestore: update(historias.estado="finalizada")
@enduml

```

6) Indicadores y Cálculo de Métricas

Agregación por **cohorte, ciudad, especialidad, médico**. Precómputo nocturno con **Functions (cron)** a `agregados`.

- **Cumplimiento calidad registro clínico** = % HC con **score** ≥ 0.9 .
- **Índice eficiencia signos vitales** = % HC con criterio(s) de signos vitales **completos** = 1.
- **Pertinencia exámenes complementarios** = % HC con criterio de pertinencia = 1.
- **Cumplimiento de guías** = % HC con adherencia = 1.
- **Cumplimiento auditorías por médico** = Auditorías realizadas / planificadas $\times 100$.
- **Cumplimiento retroalimentación** = Hallazgos con retroalimentación oportuna/documentada / Hallazgos que requieren retroalimentación $\times 100$.
- **Índices operativos**: reconsulta mensual (ver Alertas), `labs_count`, `images_count` por paciente y mes.

Esquema de `agregados.metricas`

```

{
  "total_auditas": 0,
  "cumpl_calidad": {"valor": 0.0, "meta": 0.85},
  "eficiencia_sv": {"valor": 0.0, "meta": 0.90},
  "pertinencia_ex": {"valor": 0.0, "meta": 0.90},
  "adherencia_guias": {"valor": 0.0, "meta": 0.90},
  "cumpl_por_medico": {"valor": 0.0, "meta": 0.95},
  "retro_seguimiento": {"valor": 0.0, "meta": 0.90},
  "reconsulta_mensual": 0.0,
  "labs_prom": 0.0,
  "imagenes_prom": 0.0
}

```

7) Alertas (Base Pedidos)

- **Reconsulta (7 días, misma especialidad):** ordenar pedidos por `patient_id_hash` + `especialidad` y marcar cualquier segundo evento en ventana deslizante de 7 días.
- **Umbrales:** generar alerta si `meds_count > 5`, `labs_count ≥ 5`, `images_count ≥ 2`.
- Las alertas se crean al cargar `BASE_PEDIDOS.csv` y se listan en bandeja para Admin (filtros por tipo/estado/especialidad/fecha).

Pseudocódigo reconsulta

```
for cada paciente+especialidad:
  ordenar por fecha asc
  para i=1..n:
    si fecha[i] - fecha[i-1] <= 7d -> crear alerta(reconsulta)
```

8) Seguridad y Privacidad

- **Minimización de datos:** mantener `patient_id_hash` en vez de datos personales; no almacenar nombres ni diagnósticos sensibles salvo códigos.
- **Reglas Firestore:** acceso por rol y propiedad (un auditor solo ve sus historias y auditorías, Admin ve todo); validación de esquemas a nivel cliente y Function.
- **Logs** de acciones sensibles; retención y auditoría.
- **CORS** y CSP estricta en Pages; uso de HTTPS siempre.

9) UI/UX mínima pero efectiva

- **Dashboard:** tarjetas KPI con meta/semáforo; filtros globales (mes, ciudad, especialidad, médico); gráficos por ciudad/especialidad; tabla de médicos con click → **ficha** (datos del médico, score, historias auditadas con # admisión y calificación).
- **Auditoría:** lista de criterios por secciones (acordeón), toggles 1/0, contador de completitud, atajos (1 marca sí, 0 marca no), botón "Finalizar".
- **Admin:** asistentes de carga con validación; generador de cohorte (estratificación y asignación); mantenimiento de catálogos; bandeja de alertas; bitácora.

10) Archivos de referencia (carga inicial de prueba)

- ARC mensual: [/mnt/data/ARC_AUDITORIA_MEDICA (1).csv]
- Base semanal: [/mnt/data/BASE_PEDIDOS.csv]
- Criterios (Anexo 5): [/mnt/data/Modelo Auditoría Nuevo.xlsx]
- Procedimiento institucional: [/mnt/data/MTR-GM-PR-007 Procedimiento para el manejo de auditoría médica V2.pdf]

Implementation

1) Repo y CI/CD: Vite + React + TypeScript en GitHub Pages. GitHub Actions con jobs de lint, test, build y deploy. 2) Dependencias: Firebase (Auth, Firestore, Storage, Functions), Zustand, React Router, TanStack Table, Chart.js, Papaparse, xlsx, zod, date-fns, lodash-es. 3) Estructura: módulos para auth, cohortes, historias, auditorías, admin, alertas y agregados; servicios Firebase; componentes KPI y tablas; utilidades y validaciones. 4) Reglas Firestore (borrador): Admin acceso completo; Auditor solo sus historias y auditorías; auditorías editables por su autor; lectura de catálogos y agregados para ambos perfiles. 5) Validación de datos: contratos zod para ARC_AUDITORIA_MEDICA y BASE_PEDIDOS; carga de criterios desde XLSX (Anexo 5) al catálogo de criterios. 6) Algoritmos: estratificación proporcional por especialidad con método de restos mayores y semilla por mes; asignación round-robin equilibrada por especialidad; agregación nocturna de KPIs. 7) Pantallas MVP: Login; Dashboard con filtros, KPIs y ficha de médico; Auditoría 1-clic por secciones; Admin para cargas, asignación, catálogos, alertas y bitácora. 8) Pruebas y despliegue: unitarias en utilidades, E2E con flujo completo, build a Pages y monitoreo de Functions.

Milestones

1. Semana 1: Setup, Auth, modelos, reglas, catálogo de criterios.
2. Semana 2: Carga ARC, cohorte, estratificación, asignación, bandeja auditor.
3. Semana 3: Formulario 1-clic, cierre de auditorías, agregados base.
4. Semana 4: Dashboard KPIs, ficha de médico, Base Pedidos y alertas.
5. Semana 5: Admin (catálogos, asignación manual, log), exportaciones.
6. Semana 6: Accesibilidad, seguridad, E2E y lanzamiento general.

Gathering Results

- Checklist Must-Have cumplido en demo funcional.
- Validación de KPIs contra cálculo independiente en Excel o SQL sobre muestra.
- UX: tiempo medio por historia clínica y tasa de errores de carga inferior al 1 por ciento.
- Seguridad: pruebas de reglas y revisión de minimización de datos.

Need Professional Help in Developing Your Architecture?

Please contact me at <https://sammuti.com> :)